

Autonomous Figma Product Design MCP

Product Requirements Document + Master Antigravity/Cursor Implementation Prompt

October 4, 2026

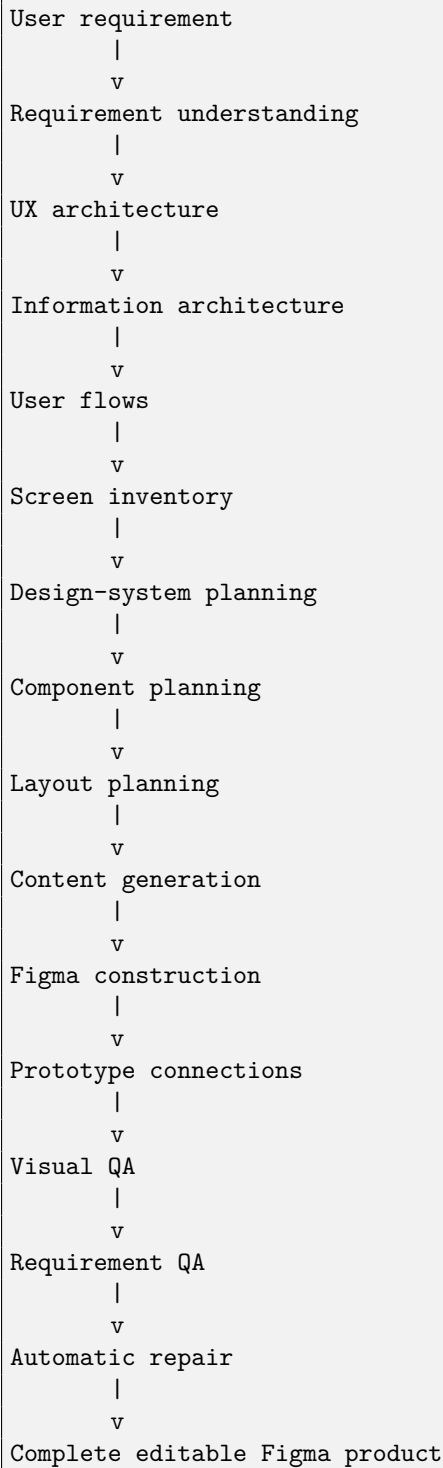
Contents

1	Executive Summary	3
2	Product Vision	4
3	Problem Statement	4
4	Goals	4
5	Non-Goals	4
6	Core User Experience	5
7	Master MCP Command	5
8	Autonomous Design Pipeline	5
8.1	Requirement Understanding	5
8.2	UX Architecture	5
8.3	Information Architecture	5
8.4	User Flow Engine	5
8.5	Screen Planning	6
8.6	Component Planning	6
8.7	Design System Engine	6
8.8	Layout Engine	6
8.9	Content Engine	6
8.10	Figma Construction	6
8.11	Prototype Construction	6
9	UI States	6
10	Existing Figma Context	6
11	Semantic Design Operations	7
12	Design Specification	7
13	Critical Architecture Rule	7
14	MCP Architecture	8
15	Recommended Technology Stack	8

16 Project Structure	8
17 Safety and Reliability	9
18 Generated Content Ownership	9
19 Visual QA	10
20 Self-Repair Loop	10
21 MVP Acceptance Criteria	10
22 Example End-to-End Requirement	10
23 Future Capabilities	10
24 Master Antigravity/Cursor Implementation Prompt	11
25 Final Product Principle	16

1 Executive Summary

The Autonomous Figma Product Design MCP is an AI-powered design engineering system that takes a natural-language product requirement and autonomously produces a complete, editable, connected, visually consistent Figma design. The user should not need to manually request individual screens, cards, buttons, layouts, flows, components, or states.



The MCP is therefore not primarily a chatbot. It is an autonomous design-build-review system.

2 Product Vision

Vision: Give the system a product requirement and receive a complete, production-oriented Figma workspace containing UX strategy, information architecture, user flows, screens, components, design system, realistic content, states, interactions, and prototype connections.

One requirement. One command. Complete product design.

3 Problem Statement

Current AI design workflows commonly require users to repeatedly instruct the system to create screens, cards, buttons, frames, connections, states, and fixes. This produces fragmented designs and places the burden of product thinking on the user. This product reverses the workflow: the AI determines what is required, constructs it, evaluates it, and repairs it.

4 Goals

1. Accept a product requirement as the primary input.
2. Understand the product and target users.
3. Derive UX architecture automatically.
4. Generate personas and Jobs-to-be-Done where useful.
5. Generate information architecture.
6. Generate complete user journeys and flows.
7. Determine required screens automatically.
8. Determine layouts automatically.
9. Determine required components automatically.
10. Generate a coherent design system.
11. Generate realistic interface content.
12. Construct actual editable Figma nodes.
13. Use Auto Layout and reusable components wherever appropriate.
14. Connect related screens into usable prototypes.
15. Generate important UI states.
16. Inspect and validate the generated design.
17. Automatically repair detected issues.
18. Preserve existing unrelated Figma content.

5 Non-Goals

The system must not produce only screenshots, flatten the product into images, execute arbitrary LLM-generated Figma code, require manual screen-by-screen prompting, invent unsupported business requirements without marking assumptions, destroy unrelated work, optimize only visual appearance, or claim completion without validation.

6 Core User Experience

The ideal interaction is:

```
design_product(  
  requirement="Build a food delivery application for college students"  
)
```

The system autonomously determines product definition, users, personas, JTBD, journeys, IA, navigation, screen inventory, components, design tokens, content, states, and prototype interactions. Subsequent commands such as “Make checkout simpler” or “Replace the home screen” modify the existing product rather than creating an unrelated design.

7 Master MCP Command

The MCP should expose one primary high-level capability: `design_product`.

```
{  
  "requirement": "string",  
  "platform": "auto | web | mobile | tablet | desktop",  
  "fidelity": "low | medium | high",  
  "style": "auto | minimal | modern | enterprise | playful | premium",  
  "autonomous": true,  
  "use_existing_figma_context": true  
}
```

Optional secondary tools may exist, but the master command must be sufficient to produce a complete first version.

8 Autonomous Design Pipeline

8.1 Requirement Understanding

Extract product purpose, target users, business objective, primary tasks, constraints, platform, entities, actions, and success criteria. Classify conclusions as **CONFIRMED**, **INFERRED**, **ASSUMPTION**, or **UNKNOWN**. Never silently convert uncertainty into fact.

8.2 UX Architecture

Generate useful personas, JTBD, user goals, pain points, key journeys, entry points, primary actions, success states, and failure states. Do not generate unnecessary artifacts merely to appear comprehensive.

8.3 Information Architecture

Determine navigation model, major sections, hierarchy, entities, relationships, screen grouping, and content hierarchy.

8.4 User Flow Engine

Build complete journeys and detect dead ends, missing transitions, missing confirmations, missing error handling, and unnecessary steps.

8.5 Screen Planning

For each flow determine the minimum complete set of screens. Each screen specification contains ID, purpose, user goal, entry conditions, exit actions, primary CTA, secondary actions, content hierarchy, components, layout, states, navigation, and prototype destinations.

8.6 Component Planning

Automatically determine reusable buttons, inputs, cards, navigation bars, sidebars, tabs, filters, tables, badges, avatars, lists, modals, dialogs, bottom sheets, forms, charts, banners, alerts, empty states, and error states.

8.7 Design System Engine

Generate or reuse color tokens, typography, spacing, grid, radius, borders, elevation, icon rules, component dimensions, and responsive rules. All screens must reference the same design-system decisions.

8.8 Layout Engine

Determine page width, container width, columns, rows, spacing, alignment, hierarchy, Auto Layout direction, component placement, and responsive behavior. Optimize for visual hierarchy and task completion rather than merely filling the canvas.

8.9 Content Engine

Generate realistic content appropriate to the product. Avoid meaningless placeholders such as Lorem ipsum unless explicitly requested.

8.10 Figma Construction

Convert the validated specification into Figma nodes. Prefer Frames, Auto Layout, Components, Variants, Instances, editable text, reusable styles/tokens, and real prototype connections.

8.11 Prototype Construction

Connect meaningful actions such as CTA-to-destination, card-to-detail, back navigation, tab navigation, submit-to-success, and cancel-to-previous-state. Do not create arbitrary connections.

9 UI States

Where applicable, automatically generate default, loading, empty, error, success, disabled, selected, focused, validation-error, permission-denied, and offline states. Only generate relevant states.

10 Existing Figma Context

When operating on an existing Figma file, inspect selected nodes, page structure, components, styles, variables/tokens when available, naming conventions, existing navigation, and nearby patterns. Reuse existing design-system conventions where appropriate and preserve unrelated content.

11 Semantic Design Operations

Internal operations should include:

```
create_product_workspace
create_page
create_section
create_screen
create_component
create_component_variant
create_instance
create_auto_layout
create_card
create_form
create_navigation
create_modal
create_table
create_list
create_empty_state
create_error_state
create_loading_state
connect_screen
apply_design_tokens
update_screen
replace_flow
duplicate_pattern
delete_generated_content
```

These operations are deterministic and schema validated.

12 Design Specification

The AI first generates a structured specification containing product, personas, flows, screens, components, design system, navigation, prototype, assumptions, and operations. It must be validated before execution.

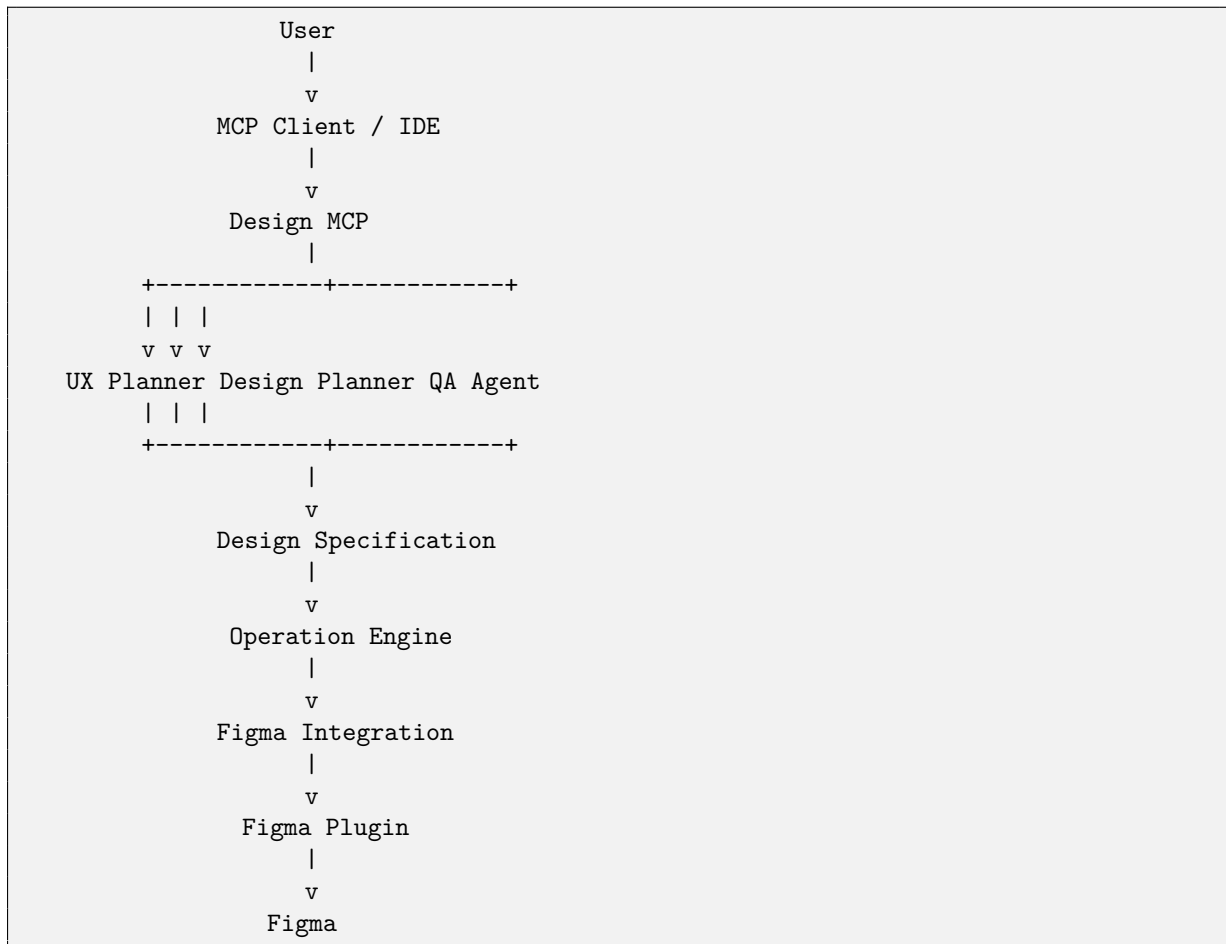
13 Critical Architecture Rule

The LLM must never directly execute arbitrary Figma API code.

```
LLM
|
v
Structured Design Specification
|
v
Schema Validator
|
v
Operation Planner
|
v
Deterministic Figma Renderer
|
v
Figma Plugin API
```

This separation is mandatory.

14 MCP Architecture



15 Recommended Technology Stack

Layer	Technology
MCP server	Python
API	FastAPI
Validation	Pydantic
AI provider	Provider abstraction with local/cloud adapters
Local AI	Ollama-compatible interface
Frontend/plugin	TypeScript + React
Figma integration	Figma Plugin API
Build	Vite
Database	PostgreSQL + pgvector
Testing	Pytest + Vitest
Containerization	Docker Compose

16 Project Structure

```
root/  
  mcp/
```

```
server.py
tools/
  design_product.py
  analyze_figma.py
  modify_product.py
schemas/
prompts/
engine/
  requirement/
ux/
  ia/
  flows/
  screens/
  components/
  design_system/
  content/
  prototype/
  qa/
renderer/
  operations/
  validation/
  planning/
plugin/
  src/
    code.ts
    ui.tsx
    api.ts
    types.ts
    figma/
  manifest.json
  package.json
tests/
docs/
data/
docker-compose.yml
.env.example
README.md
```

17 Safety and Reliability

Validate every operation and target node, prevent destructive operations by default, preserve unrelated content, support generated-content ownership markers, reject malformed specifications and arbitrary code execution, log major operations, support rollback-friendly operation groups, and distinguish facts from assumptions.

18 Generated Content Ownership

Generated nodes should have stable metadata such as product ID, generation ID, screen ID, component ID, and `managed_by=autonomous-design-mcp`. This permits safe future regeneration.

19 Visual QA

Inspect alignment, spacing, hierarchy, typography, consistency, overflow, contrast, component reuse, navigation, missing states, dead-end flows, and requirement coverage. Findings should contain severity, screen, issue, evidence, and recommended fix.

20 Self-Repair Loop

```
Generate
-> Inspect
-> Detect issues
-> Plan fixes
-> Apply fixes
-> Reinspect
-> Pass
```

Use a bounded repair count to prevent infinite loops.

21 MVP Acceptance Criteria

The MVP succeeds when a user can provide one requirement and the system can understand it, create a coherent product structure and complete primary flow, create a design system and reusable components, create real Figma frames using Auto Layout, populate realistic content, create meaningful states, connect the core prototype, inspect the result, identify and repair at least one issue, and preserve unrelated Figma content.

22 Example End-to-End Requirement

```
Build a modern food delivery app for college students.
Students should be able to discover nearby food, search restaurants,
order food, pay, and track delivery. Keep the experience fast and affordable.
```

The system should derive the appropriate product structure and generate a coherent connected design. It must not hard-code a food-delivery screen list.

23 Future Capabilities

Future versions may include multimodal input, screenshot-to-design reconstruction, existing-product redesign, design-system extraction, competitor analysis, research ingestion, analytics-informed redesign, accessibility audits, responsive generation, multi-platform generation, design-to-code, autonomous usability testing, experiment variants, and analytics feedback loops.

24 Master Antigravity/Cursor Implementation Prompt

You are the lead engineer and product architect responsible for building the Autonomous Figma Product Design MCP described in this document.

Do not build a mockup, toy, static demo, screenshot generator, or partially implemented prototype. Build the actual working system.

PRIMARY OBJECTIVE

Create an MCP-powered autonomous design engineering system where a user can provide one product requirement and the system autonomously:

1. understands the requirement;
2. identifies users and product goals;
3. derives UX architecture;
4. creates user journeys;
5. creates information architecture;
6. determines the required screens;
7. determines layouts;
8. determines components;
9. creates a coherent design system;
10. generates realistic content;
11. creates relevant UI states;
12. constructs editable Figma nodes;
13. creates meaningful prototype connections;
14. validates the resulting design;
15. detects visual and UX problems;
16. automatically repairs problems;
17. returns a completed design.

The user must NOT need to manually ask for individual screens, cards, buttons, layouts, components, or connections.

The master user-facing operation must be:

`design_product`

ARCHITECTURE

Implement these layers:

1. MCP Server
2. Requirement Intelligence
3. UX Architecture Engine
4. Information Architecture Engine
5. Flow Engine
6. Screen Planning Engine
7. Component Engine
8. Design System Engine
9. Content Engine
10. Design Specification Engine
11. Operation Validator
12. Deterministic Figma Renderer
13. Figma Plugin Integration
14. Prototype Engine
15. Visual QA Engine
16. Requirement QA Engine
17. Self-Repair Engine

Do not collapse all logic into one giant file.
Use clean modular architecture.

TECHNOLOGY

Use:

- Python
- FastAPI where HTTP APIs are useful
- Pydantic
- MCP-compatible server implementation
- provider abstraction for LLMs
- Ollama-compatible local provider
- optional cloud provider adapter
- TypeScript
- React where useful for plugin UI
- Vite
- Figma Plugin API
- Pytest
- Vitest
- PostgreSQL/pgvector when persistence is needed
- local filesystem for development artifacts

The implementation must run locally.

AI PROVIDER ABSTRACTION

Create an interface such as:

```
LLMProvider.generate_structured(...)
LLMProvider.generate_text(...)
LLMProvider.embed(...)
```

Implement a local provider first. The rest of the system must not depend directly on Ollama internals.

CRITICAL RULE

Never allow the LLM to generate arbitrary Figma API code and execute it.
The LLM generates a structured Design Specification. Then:
Design Specification -> Pydantic validation -> operation planning ->
deterministic renderer -> Figma Plugin API.

MASTER DESIGN PIPELINE

Requirement

- > Requirement Analysis
- > UX Plan
- > IA Plan
- > Flow Plan
- > Screen Plan
- > Component Plan
- > Design System
- > Content
- > Design Specification
- > Validation
- > Figma Operations
- > Render
- > Prototype

-> QA
-> Repair
-> Final Result

REQUIREMENT ENGINE

Extract product purpose, target users, business objective, primary tasks, constraints, platform, entities, actions, and success criteria.
Classify extracted information as CONFIRMED, INFERRED, ASSUMPTION, or UNKNOWN.
Do not silently hallucinate business requirements.

UX ENGINE

Generate useful users, goals, primary journey, secondary journeys, pain points, success states, failure states, and entry points. Do not generate unnecessary artifacts merely to appear comprehensive.

IA ENGINE

Determine navigation, hierarchy, sections, entities, relationships, and screen groups.

FLOW ENGINE

Generate complete end-to-end flows and detect dead ends, missing destinations, missing confirmations, missing error handling, and unnecessary steps.

SCREEN ENGINE

For each required screen create a unique screen ID, purpose, user goal, entry condition, exit actions, primary CTA, secondary actions, content hierarchy, components, layout specification, states, and prototype connections.

Do not hard-code a fixed list of screens. Derive them from the requirement.

COMPONENT ENGINE

Create reusable definitions for Button, Input, Search, Card, List, Grid, Navbar, Sidebar, Tabs, Badge, Avatar, Modal, Dialog, Bottom Sheet, Form, Table, Banner, Alert, Loading, Empty State, and Error State as applicable.

DESIGN SYSTEM ENGINE

Generate colors, typography, spacing, radii, borders, elevation, grids, and component dimensions. All screens must use the generated system.

LAYOUT ENGINE

Determine page/container width, columns, rows, spacing, alignment, hierarchy, Auto Layout direction, component sizing, and responsive behavior.
Prioritize usability and hierarchy.

CONTENT ENGINE

Generate realistic domain-specific content. Do not fill production designs with generic placeholders.

FIGMA RENDERER

Implement deterministic renderers for pages, sections, frames, text, image placeholders, components, variants, instances, buttons, inputs, cards, lists, grids, navigation, modals, forms, tables, badges, states, and prototype relationships. Prefer Auto Layout, reusable components, and editable text.

DESIGN OPERATIONS

Create typed semantic operations such as:

```
create_page
create_section
create_screen
create_auto_layout
create_component
create_component_variant
create_instance
create_card
create_button
create_input
create_navigation
create_list
create_grid
create_form
create_modal
create_table
create_state
connect_screen
apply_design_tokens
update_screen
replace_flow
delete_generated_content
```

Every operation must be schema validated.

GENERATED NODE OWNERSHIP

Every generated node should have metadata identifying product, generation, screen, component, and ownership. This enables safe future regeneration.

EXISTING FIGMA CONTEXT

When a Figma file already contains content, inspect relevant selection, existing components, styles, variables, naming conventions, navigation, and nearby patterns. Reuse existing patterns where appropriate. Never delete the entire page merely because rebuilding is easier.

FIGMA PLUGIN

Implement a plugin that communicates with the MCP/backend, inspects selection, reads relevant document context, creates and updates nodes, creates components, uses Auto Layout, creates prototype connections, and preserves unrelated nodes. The plugin is the deterministic execution layer.

MCP

Expose `design_product` as the master operation. Also implement useful secondary operations such as `analyze_figma`, `modify_product`, `regenerate_screen`,

regenerate_flow, and run_design_qa. The master operation must complete the full workflow.

QA

Implement post-generation QA for spacing, alignment, hierarchy, typography, consistency, overflow, missing content, missing states, dead-end flows, broken navigation, component consistency, and requirement coverage. Return structured findings with severity, screen, issue, evidence, and fix.

SELF-REPAIR

After rendering:

1. inspect;
2. find issues;
3. prioritize issues;
4. convert fixes into deterministic operations;
5. apply fixes;
6. inspect again.

Use a bounded repair loop. Never enter an infinite repair cycle.

ACCEPTANCE TEST

Test this requirement:

"Build a modern food delivery app for college students. Students should discover nearby food, search restaurants, order food, pay, and track delivery. Keep it fast and affordable."

The system must autonomously create a coherent product design. Do not hard-code food-delivery screens. Derive them.

Then test a second unrelated requirement to prove the architecture is general.

TESTING

Write unit tests for requirement parsing, schemas, operation validation, design-token generation, screen planning, flow validation, ownership metadata, renderer operations, and QA rules. Write integration tests for requirement -> design specification -> validated operations -> renderer. Mock external AI where necessary. Test important deterministic plugin logic.

ERROR HANDLING

Handle invalid requirements, incomplete requirements, malformed model output, unavailable LLM, invalid Figma operation, missing selection, unsupported Figma feature, renderer failures, and QA failures. Do not silently fail.

SECURITY

Never execute arbitrary model-generated code. Validate all structured model outputs. Do not expose secrets unnecessarily. Do not allow destructive Figma operations without validation.

DOCUMENTATION

Create README.md, ARCHITECTURE.md, DEVELOPMENT.md, MCP.md, FIGMA.md, and TESTING.md. README must explain how to run the complete system locally.

IMPLEMENTATION BEHAVIOR

Do not repeatedly ask for minor implementation decisions. Choose the simplest robust option, document the decision, and continue. Do not stop at scaffolding. Do not replace hard problems with TODO comments. If a Figma feature is not supported, isolate the limitation behind an adapter and implement the closest deterministic behavior.

DEFINITION OF DONE

The project is done only when:

- MCP starts successfully;
- design_product is callable;
- requirement analysis works;
- UX planning works;
- screen planning works;
- design specification validates;
- deterministic operations are generated;
- Figma plugin executes operations;
- editable screens are created;
- components are reusable;
- Auto Layout is used;
- prototype connections are created;
- QA executes;
- repair loop executes;
- generated content can be identified;
- unrelated Figma content is preserved;
- tests pass;
- documentation exists;
- the complete example works.

FINAL PRINCIPLE

The product is NOT:

"AI that tells designers what to do."

The product IS:

"An autonomous design engineer that takes a product requirement and builds the complete editable Figma product."

Prioritize this principle in every architectural and implementation decision.

25 Final Product Principle

User: Build this product.

MCP: Understands it, plans it, designs it, builds it, connects it, checks it, fixes it, and delivers the complete editable Figma workspace.